

AERIS

Adaptive Edge Runtime for Intelligent Synchronization

Architecture complète d'une plateforme Web offline transparente, compilée depuis le comportement du backend

Document d'architecture, de sécurité, d'industrialisation et de feuille de route

Décision structurante

L'IA analyse le code au build-time pour produire une représentation intermédiaire vérifiable. Le navigateur n'improvise pas le comportement du backend : il exécute un plan déterministe signé sur un snapshot local, puis réconcilie les opérations avec le serveur.

Version 1.0 - 2 octobre 2026

Résumé exécutif

AERIS est une architecture visant à rendre des applications Web capables de continuer à exécuter une partie de leur logique métier hors connexion sans imposer aux développeurs de réécrire manuellement chaque fonctionnalité en mode offline. Le principe n'est pas de placer un LLM entre le frontend et le backend pour "deviner" les résultats. La plateforme compile le comportement observable du backend en un ensemble d'artefacts déterministes : plans d'exécution locale, projections de données, règles de synchronisation, politiques de conflit et contrats de sécurité.

Quand l'utilisateur est en ligne, les requêtes suivent le chemin serveur normal et le navigateur maintient un snapshot local minimal, autorisé et versionné. Quand la connectivité disparaît, le runtime intercepte un endpoint connu, vérifie sa politique, exécute localement son plan sur le snapshot, journalise la mutation dans une outbox transactionnelle et restitue un résultat provisoire au frontend. Au retour du réseau, un Sync Gateway rejoue ou réconcilie l'intention en revalidant l'authentification, les invariants et les versions côté serveur.

La solution est industrialisable à condition de respecter une règle forte : AERIS ne promet pas de reproduire arbitrairement tout backend. Elle supporte explicitement un sous-ensemble dont les effets sont analysables, testables et classifiables. Les opérations impliquant paiement, secret serveur, transaction externe irréversible, autorisation fraîche ou invariant global strict restent online-only, sauf adaptation métier explicite.

Bloc	Responsabilité	Principe industriel
Behavior Compiler	Comprendre endpoint -> code -> données -> effets	Analyse statique d'abord, IA comme résolveur sémantique, jamais comme seule preuve
AERIS IR	Décrire la sémantique locale exécutable	Versionnée, signée, diffable, testable
Browser Runtime	Intercepter et exécuter localement	Déterministe; aucune génération libre de code en production
Local Data Plane	Conserver projections, versions et journal	Données minimisées, snapshots incrémentaux, stockage persistant quand disponible
Sync Gateway	Rejouer et réconcilier	Idempotence, revalidation, commit receipts, conflits explicites
Safety Policy	Décider ce qui peut être offline	Classification explicite; défaut conservateur = online-only

Positionnement

AERIS doit être pensé comme un compilateur + runtime distribué, pas comme un chatbot embarqué. La valeur vient de l'automatisation de la sémantique offline, de la preuve de parité avec le backend et de la réconciliation fiable.

Ce que le produit cherche à garantir

- Même appel frontend et même contrat de réponse, que le runtime choisisse le serveur ou l'exécution locale, dans les limites déclarées.
- Aucune demi-écriture : mutation locale + journal de synchronisation sont atomiques; la mutation serveur est idempotente et reconnue par reçu de commit.
- Une donnée locale est toujours associée à une version ou un curseur de snapshot; une réussite locale n'est jamais confondue avec un commit serveur.
- Toute opération offline est revalidée côté serveur avant de devenir canonique.
- Les artefacts de comportement sont générés en CI, testés différenciellement et signés avant publication.

Ce que le produit ne doit pas promettre

- Transformer automatiquement n'importe quel backend arbitraire en backend local sans annotation ni limite.

- Garantir qu'un snapshot local reflète le monde réel après une longue coupure réseau.
- Rendre offline des opérations dont la validité dépend d'une ressource mondiale, d'un secret serveur ou d'un système tiers non simulable.
- Faire confiance à un modèle IA pour décider seul de la sécurité ou de la cohérence transactionnelle.

Sommaire

1. Vision et problème à résoudre
 2. Exigences et invariants
 3. Décisions architecturales
 4. Architecture globale
 5. Behavior Compiler build-time
 6. AERIS IR - représentation intermédiaire
 7. Runtime navigateur
 8. Données locales, snapshots et deltas
 9. Cycle de vie d'une requête
 10. Synchronisation et réconciliation
 11. Concurrency et conflits
 12. Classification des endpoints
 13. Rôle de l'IA et taille des modèles
 14. Sécurité et confidentialité
 15. Contraintes spécifiques au navigateur
 16. Expérience développeur et CI/CD
 17. Observabilité et exploitation
 18. Stratégie de test et preuve de parité
 19. Déploiement progressif
 20. Stack technique recommandée
 21. MVP industriel
 22. Risques et contre-mesures
 23. Roadmap et équipe
 24. Exemple complet de bout en bout
 25. Critères d'acceptation et KPI
 26. Alternatives écartées
 27. Conclusion
- Annexes techniques
- Références

1. Vision et problème à résoudre

1.1 Problème

Une application Web classique dépend du serveur pour la logique métier et pour la donnée. Lorsqu'Internet disparaît, le frontend peut parfois afficher un cache, mais il ne sait pas toujours reproduire la sémantique exacte d'un endpoint : validations, lectures croisées, règles métier, transaction, génération d'identifiants, mise à jour de plusieurs entités et conditions d'autorisation. L'approche offline-first traditionnelle exige donc une conception spécifique dans le frontend, souvent dupliquée par rapport au backend.

L'objectif d'AERIS est de déplacer cette complexité vers une plateforme qui comprend les endpoints pendant le build, produit un programme local limité et maintient les projections de données nécessaires. Les développeurs continuent à écrire un backend normal; ils ajoutent uniquement des annotations ou des contrats lorsqu'une sémantique n'est pas inférable de manière sûre.

1.2 Proposition de valeur

- Réduire la quantité de logique métier dupliquée entre frontend et backend.
- Introduire l'offline de façon progressive, endpoint par endpoint, sans refonte globale de l'application.
- Fournir un comportement local vérifiable au lieu d'une approximation heuristique.
- Offrir une couche de synchronisation standard : outbox, idempotence, versions, conflits, reçus de commit.
- Rendre visible le niveau de sûreté de chaque endpoint dans un dashboard et dans la CI.

1.3 Cas d'usage cibles

Cas	Pertinence	Exemples
Très favorable	Données utilisateur et opérations locales ou différables	Formulaires, brouillons, tâches, notes, catalogues, lecture dossier, commentaires
Favorable avec spéculation	Invariant serveur vérifiable au retour réseau	Rendez-vous, réservation non garantie, changement de profil partagé
Défavorable par défaut	Effet externe ou invariant mondial immédiat	Paiement, virement, OTP, stock strict, signature légale, changement de rôle
Hors cible initiale	Backend hautement dynamique ou dépendant d'effets opaques	Stored procedures complexes, code généré dynamiquement, appels tiers non contractuels

2. Exigences et invariants

2.1 Exigences fonctionnelles

1. Intercepter les appels frontend vers des endpoints déclarés AERIS sans modifier leur API publique.
2. Choisir à l'exécution entre chemin serveur et plan local selon connectivité, politique et fraîcheur des données.
3. Exécuter localement les opérations autorisées avec des transactions atomiques sur les projections locales.
4. Restituer au frontend un résultat conforme au contrat de l'endpoint, avec métadonnée interne de statut provisoire.
5. Journaliser toute mutation locale avec operation_id, base snapshot, dépendances et empreinte de l'artefact utilisé.
6. Synchroniser les opérations au retour du réseau, dans un ordre respectant les dépendances.
7. Réconcilier identifiants locaux, valeurs calculées serveur, versions, tombstones et conflits.
8. Maintenir des snapshots et deltas de données limités au périmètre autorisé de l'utilisateur.

2.2 Invariants non négociables

Invariant A - autorité

Le serveur demeure l'unique source de vérité canonique. Une réussite locale est PROVISIONAL tant qu'un reçu serveur n'a pas été obtenu.

Invariant B - déterminisme

Le chemin local exécute un artefact compilé et versionné. Le modèle IA ne génère pas à la volée des requêtes SQL ou des règles métier dans le navigateur.

Invariant C - sécurité

Toute mutation rejouée est réauthenticée et revalidée côté serveur comme si elle venait d'un client potentiellement hostile.

Invariant D - atomicité locale

La mise à jour locale et l'ajout dans l'outbox appartiennent à la même transaction locale. Aucun état local ne doit pouvoir exister sans journal correspondant.

Invariant E - idempotence

Chaque intention logique possède un operation_id stable. Une retransmission ne doit pas dupliquer l'effet serveur.

3. Décisions architecturales

3.1 IA au build-time, pas dans le chemin critique

La version industrialisable sépare compréhension et exécution. La compréhension du code est un problème de compilation et de validation; l'exécution locale est un problème de runtime. Cette séparation permet de tester une fois un artefact, de le signer, puis de l'exécuter des milliers de fois sans demander à un LLM de réinterpréter le backend.

Décision	Raison	Conséquence
IA build-time	Contexte de code large, coûts et non-déterminisme acceptables	Modèle plus capable côté CI/serveur
Runtime déterministe	Reproductibilité, audit, performance	IR ou bytecode interprété/compilé vers WASM
Serveur autoritaire	Évite divergence et élévation de privilège	Toute opération locale reste provisoire
Support par adaptateurs	Les frameworks ont des conventions différentes	Commencer par un stack précis puis étendre
Fail closed	Ambiguïté = risque	Endpoint non prouvé -> online-only

3.2 Produit comme compilateur + runtime distribué

AERIS se compose de deux produits techniques distincts mais versionnés ensemble : (1) un Behavior Compiler exécuté en CI qui extrait la sémantique de l'application; (2) un runtime navigateur qui exécute les artefacts et un Sync Gateway serveur qui les réconcilie.

4. Architecture globale

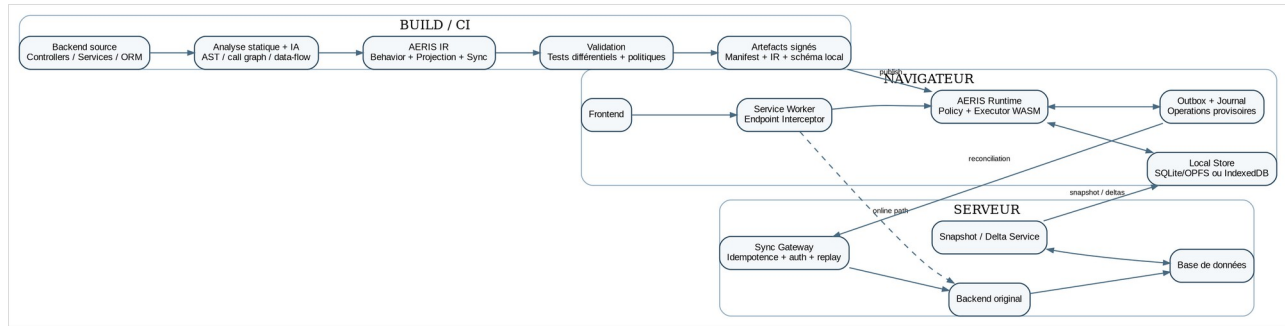


Figure 1 - Architecture générale AERIS : compilation du comportement, exécution locale et réconciliation serveur.

4.1 Flux nominal online

Le Service Worker intercepte la requête, identifie son endpoint et vérifie la version de l'artefact. Lorsque le réseau est disponible, la stratégie par défaut reste network-first pour les mutations et un mode configurable pour les lectures. La réponse serveur alimente ensuite les projections et le curseur de synchronisation.

4.2 Flux offline

Si l'endpoint est supporté et que sa politique autorise l'offline, le runtime charge l'Execution Plan correspondant, vérifie que les projections requises sont présentes et suffisamment fraîches, puis exécute la transaction locale. Le résultat est renvoyé au frontend. Une mutation est immédiatement placée dans l'outbox dans la même transaction.

4.3 Flux de retour réseau

Le moteur de synchronisation sélectionne les opérations prêtes, respecte leurs dépendances et les soumet au Sync Gateway. Le serveur revalide authentification, authorization, règles métier et versions. Il retourne un Commit Receipt ou un Conflict Package. Le navigateur applique ensuite le résultat canonique dans une transaction locale.

5. Behavior Compiler build-time

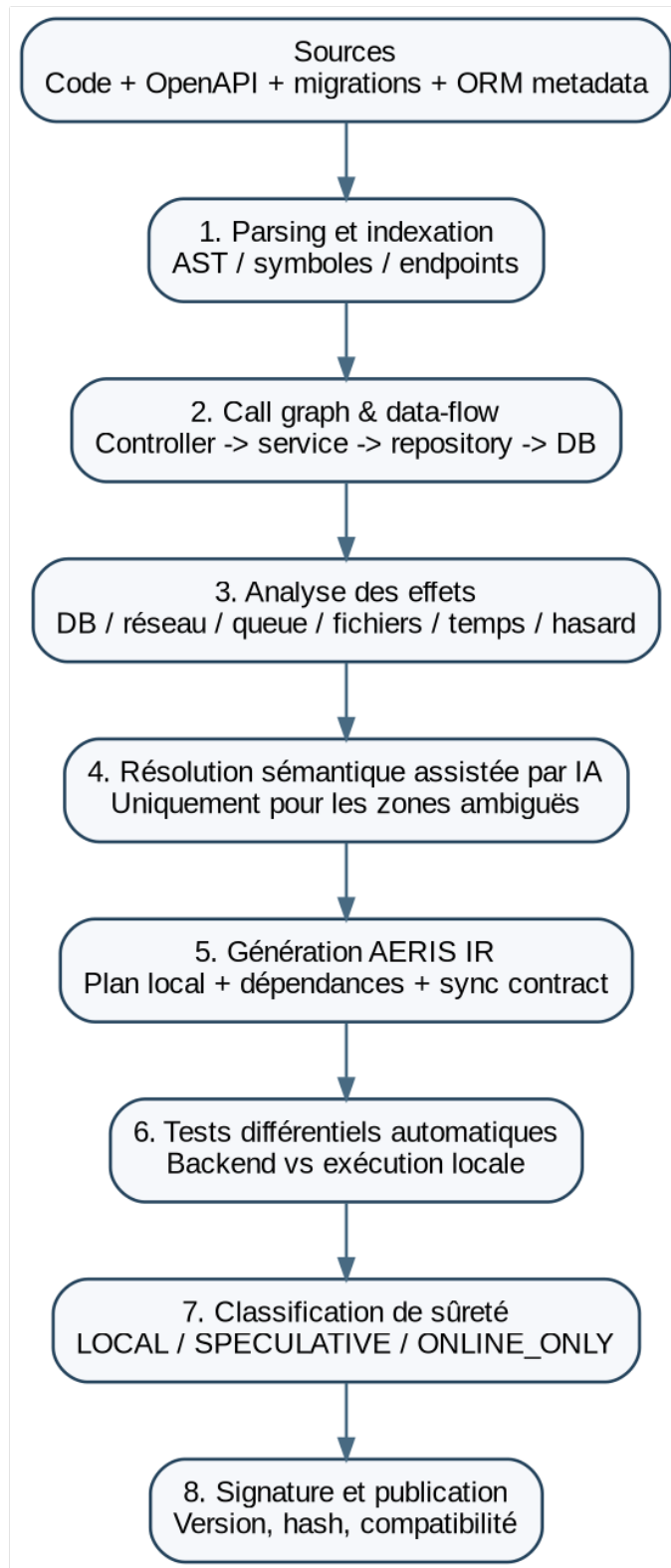


Figure 2 - Pipeline de compilation du comportement backend.

5.1 Entrées du compilateur

- Code source : controllers/routes, services, domain layer, repositories, middlewares, validators.

- Description d'API : OpenAPI 3.1.x ou métadonnées de routes générées par le framework.
- Schéma de données : migrations SQL, modèles ORM, contraintes, index, relations.
- Configuration d'injection de dépendances et mapping des adaptateurs externes.
- Annotations AERIS facultatives lorsque l'intention ou l'effet ne peut pas être prouvé automatiquement.

5.2 Analyse multi-technique

L'industrialisation ne doit pas reposer sur un LLM seul. La chaîne de preuve combine parsing AST, résolution de symboles, construction de call graph, data-flow, analyse d'effets et tests dynamiques. L'IA intervient pour comprendre des conventions métier, relier un code indirect à une intention, proposer des annotations ou expliquer une ambiguïté; sa sortie reste une proposition à vérifier.

Étape	Technique principale	Sortie
Découverte endpoint	OpenAPI + metadata framework + AST	method, path, handler, auth middleware
Call graph	Résolution statique + points-to	chaîne handler -> service -> repository
Data-flow	SSA/data-flow/taint-like	origines des inputs, champs lus, valeurs écrites
Effets	Catalogue d'adapters	DB, réseau, queue, mail, fichier, temps, random, secrets
Sémantique	LLM code + règles	conditions métier candidates et noms logiques
Validation	Tests différentiels	parité réponse + effets DB
Classification	Policy engine build-time	local-safe, speculative, online-only

5.3 Limites détectées explicitement

Le compilateur doit marquer UNSUPPORTED plutôt que d'inventer une sémantique lorsqu'il rencontre : réflexion non résolue, exécution dynamique de code, procédure stockée opaque, dépendance externe non modélisée, lecture d'un secret, fonction non déterministe non capturée, filesystem serveur, lock distribué, transaction cross-service ou événement dont l'ordre est critique.

5.4 Système d'adapters

Chaque framework supporté possède un adapter qui sait retrouver les routes, les middlewares, l'ORM et les conventions de transaction. Pour le MVP, une seule combinaison doit être supportée à fond. Exemple recommandé : NestJS + TypeORM/Prisma + PostgreSQL + React/Next.js. Le support Spring Boot/Hibernate ou Django/DRF vient ensuite.

6. AERIS IR - représentation intermédiaire

6.1 Pourquoi une IR

L'IR découple le backend original du navigateur. Elle décrit ce qui doit être lu, calculé, validé et écrit sans embarquer le code serveur ni exiger le même langage. Elle doit être suffisamment expressive pour le sous-ensemble supporté, mais volontairement limitée pour rester analysable.

6.2 Artefacts publiés

Artefact	Contenu	Consommateur
Endpoint Manifest	route, method, auth scope, policy, version	Service Worker / Router
Behavior IR	instructions, conditions, effets locaux	Local Executor

Artefact	Contenu	Consommateur
Projection Manifest	tables/champs/reliations/filters nécessaires	Snapshot Service + Local Store
Sync Contract	idempotence, base version, remapping IDs, conflits	Sync Gateway + client
Test Vectors	fixtures, expected response/effects	CI + runtime conformance
Signature Envelope	hash, key id, version compat	Loader / updater

6.3 Exemple de Behavior IR

Exemple conceptuel (format illustratif)

```

endpoint: POST /appointments
version: 31
inputs:
  patientId: uuid
  doctorId: uuid
  startAt: datetime
reads:
  - Patient[id = input.patientId]
  - Doctor[id = input.doctorId]
  - Availability[doctorId = input.doctorId, startAt = input.startAt]
assert:
  - Patient.exists
  - Doctor.exists
  - Availability.free == true
writes:
  - create Appointment(localId(), patientId, doctorId, startAt, status="PENDING_SYNC")
  - update Availability set free=false
transaction: atomic
offline_policy: SPECULATIVE
sync:
  idempotency: operation_id
  base: snapshot_cursor
  conflict: SERVER_REVALIDATE

```

6.4 Jeu d'instructions minimal

- LOAD/QUERY projection locale avec filtres bornés et index connus.
- ASSERT pour validations déterministes et invariants locaux.
- COMPUTE pour expressions pures : concaténation, arithmetic, date relative capturée, mapping enum.
- INSERT/UPDATE/DELETE sur projection locale dans une transaction.
- EMIT_LOCAL_EVENT pour mise à jour UI interne, sans effet externe.
- QUEUE_INTENT pour journaliser l'intention serveur.
- RETURN pour reconstruire le DTO contractuel.

Toute instruction qui implique réseau externe, secret serveur ou primitive non déterministe non capturée doit produire ONLINE_ONLY ou exiger une extension explicitement auditée.

7. Runtime navigateur

7.1 Composants

Composant	Rôle	Implémentation recommandée
Service Worker	Interceptor fetch, cache shell, router vers	JavaScript/TypeScript

Composant	Rôle	Implémentation recommandée
	runtime	
Endpoint Router	Matcher method/path + version	Table compilée / trie
Policy Engine	Autoriser/bloquer local selon classe, TTL, auth	Déterministe
IR Executor	Exécuter plans et expressions	Rust -> WebAssembly ou TS strict
Local DB Adapter	Transactions, requêtes, indexes	SQLite WASM+OPFS; IndexedDB fallback
Snapshot Manager	Initial snapshot, deltas, tombstones	Worker dédié
Outbox	Journal durable des opérations	Même DB que projections
Sync Engine	Ordonnancement, retry, backoff, receipts	Worker / service worker
Tab Coordinator	Eviter deux sync simultanées	Web Locks + fallback leader election
Artifact Loader	Télécharger, vérifier signature, activer version	Cache + cryptographic hash

7.2 Séparation des threads

L'UI ne doit pas exécuter les analyses ou transactions lourdes. Le Service Worker intercepte et délègue; un Worker dédié héberge idéalement SQLite/WASM et l'exécuteur. L'UI reçoit uniquement les résultats. Cela limite les blocages du main thread et respecte les contraintes d'OPFS, dont les modes performants sont orientés Worker.

7.3 Compatibilité et fallback

Le runtime doit fonctionner sans Background Sync. Cette API n'est pas disponible partout; elle est donc une optimisation, pas un prérequis. Les triggers de synchronisation de base sont : retour online, ouverture/reprise de l'application, action utilisateur et timer pendant que l'app est active. Background Sync est utilisé lorsqu'il existe.

8. Données locales, snapshots et deltas

8.1 Ne jamais cloner toute la base serveur

Le navigateur reçoit une projection utilisateur minimale, dérivée des endpoints offline réellement activés. La projection limite lignes et colonnes selon les autorisations. Les données sensibles inutiles, secrets, logs internes, tokens, colonnes techniques et enregistrements hors périmètre ne doivent jamais être synchronisés.

8.2 Snapshot initial

```
SnapshotEnvelope {
  projection_version: "proj-42",
  cursor: 981244,
  user_scope_hash: "...",
  generated_at: "2026-10-02T12:00:00+01:00",
  entities: {...},
  tombstones: [...],
  artifact_version: "aeris-31"
}
```

Le snapshot est validé et écrit de façon atomique. Une projection partiellement téléchargée n'est jamais activée. Le runtime conserve au minimum la version active et, si l'espace le permet, une version précédente pour rollback.

8.3 Deltas

Après le snapshot, le client demande seulement les changements depuis son curseur. Chaque delta transporte la version de l'enregistrement, l'opération (upsert/delete), la projection concernée et une preuve de compatibilité d'artefact. Les suppressions utilisent des tombstones afin qu'un client offline ne ressuscite pas une entité supprimée.

8.4 Fraîcheur

Classe de donnée	TTL indicatif	Comportement hors TTL
Référentiel quasi statique	heures/jours	lecture locale possible avec badge de fraîcheur
Profil utilisateur	minutes/heures	écriture locale possible si versionnée
Disponibilité/réservation	secondes/minutes	spéculatif uniquement
Prix/stock strict	très court	online-only
Autorisation/roles	très court	ne jamais accorder un privilège nouveau offline

8.5 Gestion du quota navigateur

Le runtime interroge le quota et l'usage, demande un stockage persistant lorsque disponible et applique des priorités d'éviction internes. L'outbox et les métadonnées de commit sont prioritaires; les snapshots reconstruisibles peuvent être évincés. En cas de pression, le runtime réduit les projections ou désactive temporairement certaines capacités offline au lieu de risquer la perte du journal.

9. Cycle de vie d'une requête

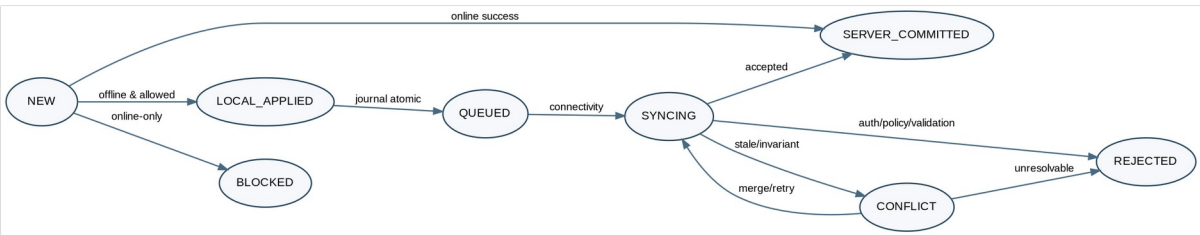


Figure 3 - Machine d'état d'une opération AERIS.

9.1 Etats

Etat	Sens	Visible utilisateur
NEW	requête interceptée, non décidée	non
LOCAL_APPLIED	effet appliqué localement	oui, résultat disponible
QUEUED	mutation présente dans l'outbox	éventuellement badge "à synchroniser"
SYNCING	transmise au serveur	généralement non
SERVER_COMMITTED	effet accepté et canonique	oui, état normal
CONFLICT	serveur refuse la base ou invariant	oui si arbitrage nécessaire
REJECTED	validation/auth/policy invalide	oui, rollback ou correction

Etat	Sens	Visible utilisateur
BLOCKED	opération online-only hors ligne	oui, connexion requise

9.2 Réponse au frontend

Le frontend doit idéalement conserver le même contrat métier. AERIS ajoute des métadonnées hors payload métier via headers ou canal runtime, par exemple X-AERIS-State: provisional, afin que l'application puisse afficher un indicateur sans obliger toutes les features à gérer un modèle de données différent.

9.3 Identifiants

Les créations offline utilisent des identifiants client globalement uniques (UUIDv7/ULID selon choix). Si le backend impose des IDs séquentiels, le Commit Receipt renvoie une table de remapping local_id -> server_id. Le runtime met à jour toutes les références locales dans une transaction.

10. Synchronisation et réconciliation

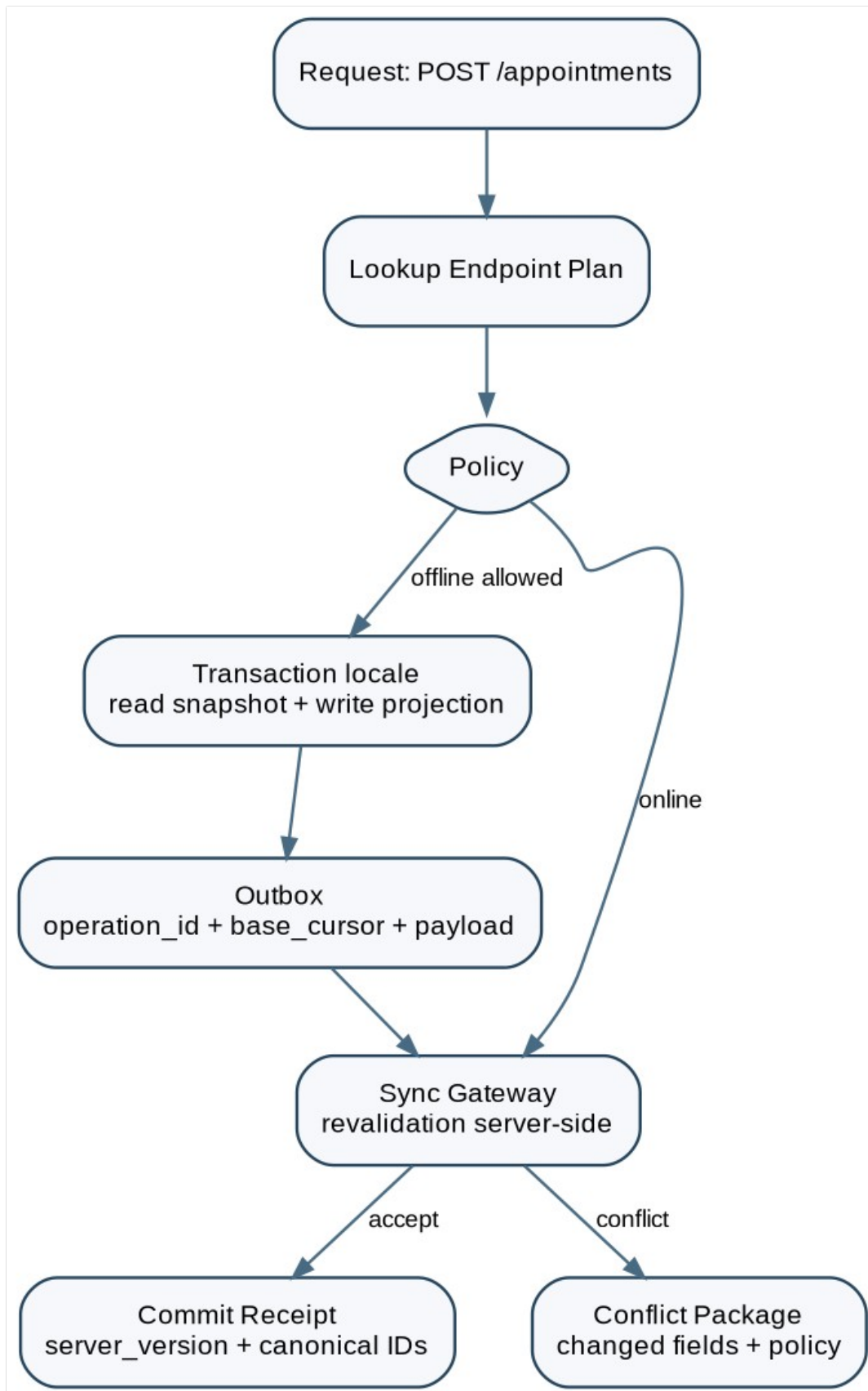


Figure 4 - Exécution locale et passage par le Sync Gateway.

10.1 Outbox transactionnelle

Toute mutation locale écrit dans la même transaction : (a) le nouvel état de projection; (b) l'operation_id; (c) la route logique; (d) le payload normalisé; (e) le cursor de base; (f) les dépendances; (g) la version d'IR; (h) le hash de l'état préconditionnel utile à la détection de conflit.

10.2 Sync Gateway

Le Sync Gateway est préférable à un simple replay direct contre tous les endpoints, car il centralise l'idempotence, les receipts, la vérification de version d'artefact et la gestion des batches. Il peut ensuite appeler le backend original via son API publique ou une interface interne dédiée.

10.3 Reçu de commit

```
CommitReceipt {
  operation_id: "01J...",
  status: "COMMITTED",
  committed_at: "...",
  server_cursor: 981311,
  canonical_response: {...},
  id_map: {"local-...": "srv-..."},
  changed_records: [...],
  replay_hash: "..."}
}
```

10.4 Retries

Les retries utilisent backoff exponentiel avec jitter, mais l'operation_id reste stable. Les erreurs sont classées en retryable (réseau, 5xx temporaire), conflit (409/412 ou invariant) et terminal (auth révoquée, validation métier définitive). Un retry ne doit jamais modifier silencieusement l'intention initiale.

11. Concurrency et conflits

11.1 Principe

AERIS ne cherche pas à supprimer les conflits; il cherche à les détecter tôt, les classer correctement et appliquer une politique explicite. Toute mutation possède une base versionnée. Le serveur compare cette base à l'état canonique.

11.2 Stratégies

Stratégie	Quand l'utiliser	Exemple
No conflict / append	Opération commutative ou append-only	ajout de commentaire avec ID unique
Field merge	Champs indépendants modifiés séparément	préférences UI vs numéro de téléphone
Optimistic version check	Objet partagé avec version	édition d'un profil
Server revalidate	Invariant dépend du monde actuel	créneau de rendez-vous
CRDT ciblé	Document collaboratif compatible CRDT	texte/notes partagées
User arbitration	Conflit sémantique non fusionnable	deux valeurs métier incompatibles
Reject + compensate	Effet local provisoire invalide	réservation perdue au retour réseau

11.3 Dépendances entre opérations

L'outbox forme un DAG, pas une simple FIFO. Une tâche créée offline puis commentée dépend de l'opération de création. Le scheduler ne synchronise le commentaire qu'après résolution de l'ID canonique de la tâche. Les cycles doivent être détectés au build ou transformés en transaction groupée.

11.4 Multi-device

Deux appareils offline d'un même utilisateur peuvent diverger. Le serveur arbitre à la synchronisation. Un appareil ne doit jamais supposer qu'il est le seul client; les versions serveur et les politiques de merge restent obligatoires.

12. Classification des endpoints

Le Behavior Compiler attribue une classe de sûreté. La classification est visible dans le rapport CI et peut être surchargée par une annotation validée par revue de code.

Classe	Offline	Critères	Exemples
LOCAL_READ_SAFE	Oui	lecture sur projection, pas d'effet	GET profil, liste locale
LOCAL_WRITE_SAFE	Oui	effet local déterministe, invariant local	brouillon, préférence locale synchronisable
REPLAYABLE	Oui différé	serveur peut appliquer idempotement	commentaire, création d'objet non exclusif
SPECULATIVE	Oui provisoire	résultat peut être invalidé par concurrence	RDV, réservation non garantie
ONLINE_REQUIRED	Non	secret, invariant global, effet externe irréversible	paiement, rôle, OTP, signature
UNSUPPORTED	Non	analyse non concluante ou primitive inconnue	réflexion, stored procedure opaque

12.1 Score de preuve interne

Un score interne peut aider le compilateur à décider si une analyse est suffisante, mais il ne doit pas devenir un "score de sécurité" marketing. Les facteurs : couverture call graph, proportion d'effets connus, couverture tests différentiels, déterminisme des expressions, absence de dépendances opaques. En dessous du seuil, la classe retombe automatiquement à ONLINE_REQUIRED ou UNSUPPORTED.

13. Rôle de l'IA et taille des modèles

13.1 Où l'IA apporte réellement de la valeur

- Résoudre des chaînes d'appels ou conventions métier difficiles à nommer à partir des AST seuls.
- Interpréter des validations métier dispersées et proposer une normalisation dans l'IR.
- Générer des tests différentiels et des fixtures ciblées à partir des branches de code.
- Expliquer pourquoi un endpoint est classé online-only et proposer une annotation ou refactor pour le rendre analysable.
- Assister la génération d'adapters pour un nouveau framework, sans les activer automatiquement.

13.2 Taille recommandée

Emplacement	Taille indicative	Rôle
CI local / self-hosted léger	7B-14B code spécialisé	analyse sémantique secondaire, génération

Emplacement	Taille indicative	Rôle
		de tests
CI plus robuste	14B-32B code spécialisé	meilleure compréhension inter-fichiers et refactors
Cloud build service	modèle plus grand selon politique	cas complexes, fallback contrôlé
Navigateur production	0 LLM requis	exécution IR déterministe
Navigateur optionnel	petit modèle 100M-500M max	diagnostic/UX local, jamais source de vérité

Le besoin initial d'un modèle embarqué disparaît donc presque entièrement dans l'architecture industrielle. C'est un avantage : pas de téléchargement de centaines de Mo, pas de dépendance WebGPU, pas de latence d'inférence pour chaque requête, et surtout une sémantique stable.

13.3 Garde-fous IA

- La sortie IA est sérialisée dans un schéma strict; aucune sortie libre n'est directement exécutée.
- Chaque instruction IR doit être reliée à des preuves : emplacement de code, contrainte de schéma ou annotation.
- Les tests différentiels doivent passer avant publication.
- Une version de modèle ou de prompt différente produit un nouveau build d'artefacts, jamais une mutation silencieuse en production.

14. Sécurité et confidentialité

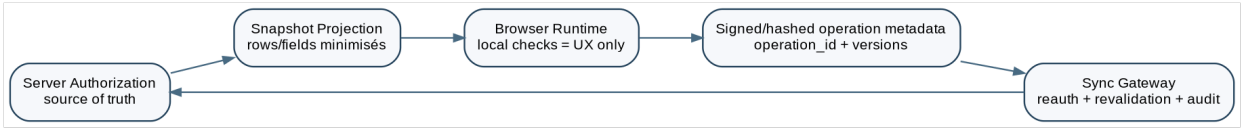


Figure 5 - Modèle de confiance : le navigateur accélère l'expérience, le serveur reste autoritaire.

14.1 Modèle de menace

Le navigateur est un environnement non fiable du point de vue serveur. Un utilisateur peut modifier IndexedDB, patcher le JavaScript, appeler le Sync Gateway directement ou falsifier l'outbox. AERIS ne doit donc utiliser aucune donnée locale comme preuve d'autorisation ou de vérité serveur.

14.2 Mesures

Risque	Mesure
Altération artefact	signature Ed25519/ECDSA, hash de contenu, version pinning
Rejeu malveillant	operation_id + registre idempotent + user binding
Elévation de privilège	revalidation RBAC/ABAC côté serveur; snapshot ne donne jamais un droit
Exfiltration locale	minimisation projection, CSP stricte, Trusted Types, réduction XSS
Vol appareil	session expirante; données minimisées; chiffrement local optionnel selon modèle de menace
Fuite inter-utilisateur	partition par user/tenant et purge atomique au logout
Downgrade artefact	minimum supported version et refus des manifestes obsolètes

Risque	Mesure
Poisoning du snapshot	TLS + signature/empreinte d'enveloppe + validation schema
Sync forgée	serveur reconstruit et revalide l'intention, ne fait pas confiance au résultat local

14.3 Chiffrement local : nuance importante

WebCrypto peut protéger certaines données au repos, mais un XSS actif dans la même origine peut souvent accéder au code et aux clés disponibles à l'application. Le premier contrôle reste donc la minimisation, la protection contre XSS et la réduction de la durée de rétention. Le chiffrement local est utile contre certains scénarios de récupération de fichiers ou de perte d'appareil, mais il ne remplace pas l'isolation de l'origine.

14.4 Données réglementées

Pour santé, finance ou données fortement réglementées, les projections offline doivent être explicitement approuvées par type de donnée. Il faut prévoir politiques de rétention, purge distante au prochain contact, journal d'accès, classification PII/PHI et contraintes de résidence selon le contexte juridique réel du produit.

15. Contraintes spécifiques au navigateur

15.1 Service Worker

Le Service Worker est adapté à l'interception des requêtes et aux expériences offline, mais son cycle de vie est contrôlé par le navigateur. Il peut être arrêté. AERIS ne doit donc pas dépendre d'un processus toujours vivant : toute progression de sync est persistée et reprise idempotemment.

15.2 Background Sync

Background Sync est utile lorsqu'il est disponible mais sa compatibilité reste incomplète. Le produit doit garantir la synchronisation via des triggers de premier rang : événement online, démarrage/reprise, focus d'application et action utilisateur. Le background sync est un bonus.

15.3 Stockage

IndexedDB est largement disponible et transactionnel. SQLite/WASM avec OPFS apporte une sémantique SQL et des transactions plus proches du backend, mais doit être encapsulé dans un adapter. Le runtime vérifie le quota, demande la persistance et gère l'éviction interne. Le journal d'opérations est plus important que le cache de lecture.

15.4 Multi-onglets

Deux onglets peuvent déclencher deux synchronisations. Web Locks fournit une coordination intra-origin sur les navigateurs compatibles. Un fallback peut utiliser BroadcastChannel + lease persisté. Le lock ne remplace pas l'idempotence serveur : il réduit seulement les duplications locales.

15.5 Mise à jour de version

Un nouveau frontend peut être incompatible avec un ancien manifest ou une ancienne projection. AERIS définit une matrice de compatibilité : app_version, artifact_version, projection_version. Une migration locale est atomique; en cas d'échec, le runtime conserve l'ancienne version ou force une resynchronisation avant de réactiver l'offline.

16. Expérience développeur et CI/CD

16.1 Philosophie

Le développeur ne doit pas écrire “if offline” dans chaque feature. Il écrit un backend normal. AERIS analyse et produit un rapport. Il intervient uniquement lorsque la CI signale qu’une logique est ambiguë, non déterministe ou non sûre.

16.2 CLI

```
aeris init
aeris analyze
aeris explain POST /appointments
aeris test --differential
aeris diff --against main
aeris publish --environment staging
aeris report --format html
```

16.3 Annotations / extensions

Exemple conceptuel côté framework

```
@AerisOffline(policy = SPECULATIVE)
@AerisProjection(fields = {"id", "doctorId", "startAt", "status"})
@AerisConflict(strategy = SERVER_REVALIDATE)
@AerisExternalEffect(onlineOnly = true)
```

L'équivalent OpenAPI peut utiliser des extensions x-aeris-* pour les équipes qui préfèrent un contrat API déclaratif. Les annotations ne doivent pas permettre de contourner une interdiction critique sans règle de revue et tests supplémentaires.

16.4 Quality gates CI

Gate	Condition d'échec
Coverage endpoint	endpoint offline sans handler entièrement résolu
Effect audit	effet inconnu ou externe non déclaré
Differential tests	réponse ou effets divergents au-delà de la tolérance
Schema compatibility	migration locale non définie ou destructrice
Security	projection contient champ interdit ou scope non prouvé
Performance	plan local dépasse budget CPU/mémoire/latence
Artifact signing	artefact non signé ou dépendance non reproductible

17. Observabilité et exploitation

17.1 Identifiants de corrélation

Chaque opération possède operation_id. Les appels online et les replays transportent aussi traceparent lorsque l'infrastructure utilise W3C Trace Context. Cela permet de relier une action utilisateur, son exécution locale, son replay et le commit backend dans la même investigation.

17.2 Métriques

Métrique	But
offline_execution_rate	part des endpoints exécutés localement
local_p50/p95_latency	qualité UX hors ligne
sync_lag_seconds	temps entre intention locale et commit serveur
conflict_rate_by_endpoint	identifier les features mal adaptées au offline
rejection_rate	erreurs métier/auth après reprise
outbox_depth/age	détecter blocage de synchronisation
snapshot_size / quota_ratio	prévenir pression de stockage
differential_mismatch_rate	parité compilateur/backend
artifact_version_distribution	contrôler rollouts et versions anciennes

17.3 Logs

Les logs navigateur ne doivent pas contenir les payloads sensibles par défaut. Le runtime journalise codes de transition, versions, hashes et catégories d'erreur. Le diagnostic détaillé est opt-in et redacted. Le serveur conserve l'audit de réconciliation selon la politique du produit.

18. Stratégie de test et preuve de parité

18.1 Test différentiel - pièce maîtresse

Pour un endpoint offline, la CI crée un état de base identique pour deux exécuteurs : backend réel dans un environnement isolé et AERIS Executor sur projection locale. Elle envoie le même input et compare : code de réponse logique, DTO, enregistrements lus/écrits, contraintes, événements locaux autorisés et erreurs. Toute différence non explicitement tolérée bloque la publication.

18.2 Niveaux de tests

1. Unit tests de l'IR Executor et des adapters de stockage.
2. Golden tests par endpoint avec fixtures versionnées.
3. Property-based tests pour valeurs limites et invariants.
4. Mutation testing du backend pour vérifier que les tests différentiels détectent une divergence.
5. Chaos réseau : coupure avant/après requête, duplication, latence, reprise, réordonnancement.
6. Tests multi-tab et multi-device.
7. Tests quota/eviction et migration de projection.
8. Cross-browser : Chromium, Firefox, Safari selon support produit.
9. Security tests : falsification outbox, stale token, downgrade, XSS, replay.

18.3 Shadow execution

Avant d'autoriser une écriture offline, le runtime peut fonctionner en shadow mode lorsque l'utilisateur est online : il exécute localement sans impacter l'UI, appelle le serveur normalement puis compare résultat et effets dérivables. Cela fournit une télémétrie de parité sur des usages réels sans risquer la donnée canonique.

19. Déploiement progressif

Phase	Capacité	Critère de passage
0 - Analyse seule	rapport endpoints et projections	couverture call graph stable
1 - Read shadow	exécution locale invisible en online	mismatch très faible et compris
2 - Offline read	lectures depuis snapshot	fraîcheur/UX acceptables
3 - Safe writes	brouillons, append, préférences	idempotence et outbox éprouvées
4 - Speculative writes	invariants revalidés serveur	conflicts maîtrisés + UX de compensation
5 - Extension framework	second stack backend	adapter conformance suite passée

19.1 Feature flags

Chaque endpoint est activable par environnement, tenant, cohorte et navigateur. Un kill switch serveur peut désactiver l'exécution locale d'un endpoint sans attendre un nouveau déploiement frontend, en publiant une policy manifest plus restrictive.

20. Stack technique recommandée

Couche	Choix recommandé initial	Pourquoi
Compiler core	Rust ou TypeScript + modules spécialisés	performance, tooling, intégration CI
Parsing multi-langage	Tree-sitter + AST natifs framework	parsing robuste, grammaires nombreuses
Data-flow avancé	CodeQL/Semgrep-like ou analyse maison ciblée	call/data flow sans dépendre de l'IA
LLM build-time	modèle code 7B-32B self-hosted ou service contrôlé	sémantique inter-fichiers et génération tests
IR schema	JSON Schema/Protobuf + canonical encoding	validation et versioning
Executor	Rust -> WebAssembly	performance et sandbox logique
Browser DB	SQLite WASM + OPFS; IndexedDB fallback	transactions SQL et portabilité
Interception	Service Worker	proxy réseau offline natif Web
Coordination	Web Locks + BroadcastChannel fallback	multi-onglets
Server sync	service stateless + idempotency store	scalable et indépendant backend
Delta feed	CDC ou application change log	snapshots incrémentaux
Tracing	OpenTelemetry/W3C Trace Context	corrélation bout en bout
Artifacts	registry signé + CDN	déploiement versionné et rollback

20.1 Stack MVP conseillée

Pour réduire le risque, commencer avec une seule verticale technique : React/Next.js côté Web, NestJS côté backend, PostgreSQL et Prisma ou TypeORM. Cette combinaison facilite la découverte de routes et le mapping ORM. Le premier objectif n'est pas le multi-framework; c'est la preuve de parité et la fiabilité du cycle local -> sync -> commit.

21. MVP industriel

21.1 Scope

- Support GET simple, POST create et PATCH update pour une application CRUD réaliste.
- Projection de 5 à 10 entités, relations simples 1-N/N-1, contraintes de base.
- Classes LOCAL_READ_SAFE, REPLAYABLE et SPECULATIVE.
- SQLite/WASM+OPFS sur navigateurs compatibles et fallback IndexedDB.
- Outbox, idempotency, receipts, remapping d'IDs, optimistic versioning.
- Compiler NestJS + Prisma/TypeORM.
- Tests différentiels automatisés sur chaque endpoint activé.
- Dashboard de compatibilité endpoint et shadow mode.

21.2 Hors scope MVP

- Transactions distribuées multi-services.
- Stored procedures arbitraires.
- Paiement et effets tiers irréversibles.
- Support universel de tous frameworks.
- CRDT générique pour toutes les entités.
- LLM exécuté dans le navigateur pour la sémantique métier.

21.3 Démo de référence

Construire une application de gestion de tâches/projets avec utilisateurs, projets, tâches, commentaires, pièces jointes métadonnées et assignations. Elle doit démontrer : création offline, dépendances (commentaire sur tâche créée offline), édition concurrente sur deux navigateurs, reprise réseau, conflit résolu, quota limité, migration de schéma et shadow mode.

22. Risques et contre-mesures

Risque	Impact	Contre-mesure
Analyse statique incomplète	IR incorrecte	scope framework étroit, fail-closed, preuves source, tests différentiels
LLM hallucine une règle	sémantique fausse	sortie non exécutable directement, validation structurelle + tests
Snapshot obsolète	résultat local invalide	classe SPECULATIVE, TTL, server revalidate
Perte stockage navigateur	perte du cache/journal	persist(), journal prioritaire, sync fréquente, UI d'alerte
XSS	exfiltration données locales	CSP, Trusted Types, supply-chain security, minimisation
Deux onglets sync	duplication	Web Locks + idempotence serveur
Migration cassée	offline indisponible	double-buffer, rollback, resnapshot
Explosion taille snapshot	UX/stockage	projection par feature, pagination, quotas,

Risque	Impact	Contre-mesure
		téléchargement adaptatif
Conflits trop fréquents	expérience trompeuse	désactiver offline pour endpoint ou revoir sémantique
Couplage fort backend	maintenance adapter	IR stable + adapters versionnés + conformance suite

23. Roadmap et équipe

23.1 Roadmap indicative

Étape	Durée indicative	Livrable
R&D architecture	3-5 semaines	IR v0, threat model, stack cible, démonstrateur intercept
PoC compilateur	5-8 semaines	NestJS routes -> call graph -> IR pour CRUD
Runtime local	5-8 semaines	SQLite/IndexedDB, executor, outbox
Sync Gateway	4-6 semaines	idempotence, receipts, version conflicts
Differential testing + shadow	4-6 semaines	preuve de parité automatisée
Alpha intégrée	6-10 semaines	application pilote, dashboard, hardening
Production hardening	8-16+ semaines	sécurité, perf, compat, SRE, migrations

Ces durées sont des ordres de grandeur et se chevauchent. Une équipe expérimentée peut paralléliser compiler, runtime et sync. Le facteur dominant est la variété de sémantique backend à supporter, pas le code du Service Worker.

23.2 Equipe minimale

- 1 architecte/tech lead systèmes distribués et cohérence.
- 1 ingénieur compiler/static analysis.
- 1 ingénieur Web/WASM/storage navigateur.
- 1 ingénieur backend/synchronisation/PostgreSQL.
- 1 ingénieur QA/SDET orienté differential/chaos tests (peut être partagé au début).
- Appui sécurité et product/UX pour les statuts provisoires et conflits.

24. Exemple complet de bout en bout

24.1 Backend

```
POST /appointments
Controller -> AppointmentService.create()
-> PatientRepository.findById(patientId)
-> DoctorRepository.findById(doctorId)
-> AvailabilityRepository.findSlot(doctorId, startAt)
-> assert(slot.free)
-> AppointmentRepository.insert(...)
-> AvailabilityRepository.update(slot, free=false)
-> return AppointmentDto
```

24.2 Compilation

Le compilateur découvre les trois lectures, les deux écritures et la transaction. Il détecte que l'invariant "slot.free" dépend d'une donnée modifiable par d'autres utilisateurs. L'endpoint n'est donc pas LOCAL_WRITE_SAFE; il devient SPECULATIVE. La projection contient uniquement les champs nécessaires de Patient, Doctor, Availability et Appointment.

24.3 Dernier état online

Le navigateur possède cursor=981244. Le créneau D42/10:00 est libre dans le snapshot. Le runtime connaît artifact_version=31 et projection_version=42.

24.4 Coupure réseau

L'utilisateur envoie POST /appointments. Le Service Worker matche l'endpoint. La policy autorise SPECULATIVE car le snapshot a moins de 10 minutes. L'exécuteur ouvre une transaction locale, vérifie Patient/Doctor/Availability, crée Appointment local UUID, marque le slot non libre et insère l'opération dans l'outbox. Le frontend reçoit 201 avec le DTO attendu; l'état interne est provisional.

24.5 Concurrency

Pendant la coupure, un autre utilisateur réserve le même créneau sur le serveur. Le serveur passe à cursor=981300.

24.6 Reconnexion

AERIS envoie l'operation_id, le payload, base_cursor=981244 et les préconditions. Le serveur revalide l'endpoint sur l'état courant, constate que le slot est occupé et retourne Conflict Package. La policy de l'endpoint exige SERVER_REVALIDATE et n'autorise aucun merge.

24.7 Compensation

Le runtime annule l'Appointment provisoire ou le marque rejected selon UX, restaure la projection du slot à partir du delta serveur et notifie le frontend que le créneau n'est plus disponible. L'utilisateur peut choisir un autre créneau. Aucune réservation serveur fantôme n'a été créée, et aucun double effet ne s'est produit.

24.8 Variante sans conflit

Si le créneau est encore libre, le serveur commit l'opération, renvoie le server_cursor et éventuellement un ID canonique. Le runtime remappe l'ID, applique le receipt, retire l'opération de l'outbox et passe l'état à SERVER_COMMITTED.

25. Critères d'acceptation et KPI

25.1 Critères fonctionnels MVP

- Un endpoint activé fonctionne en ligne sans changement de contrat frontend.
- Le même endpoint continue hors ligne si sa classe le permet.
- La mutation locale et l'outbox sont atomiques même en crash simulé.
- Une opération rejouée 10 fois ne produit qu'un seul effet serveur.
- Un conflit de version est détecté et ne devient jamais un faux commit.
- Le logout purge les projections de l'utilisateur et bloque le replay sans nouvelle auth.
- Une migration d'artefact incompatible force resnapshot ou fallback online sans corruption.

25.2 SLO techniques proposés pour pilote

Indicateur	Cible pilote
Parité differential tests	100% des endpoints activés sur suite de référence
p95 exécution locale CRUD simple	< 100 ms hors rendu UI sur machine cible
Double-commit serveur	0 toléré
Outbox corruption après crash tests	0 toléré
Endpoints offline avec effet inconnu	0
Taux de conflits SPECULATIVE	mesuré; seuil métier par endpoint
Rollback artefact	automatique et testé
Compatibilité navigateur	matrice explicitement publiée

26. Alternatives écartées

Alternative	Pourquoi ne pas la retenir comme architecture centrale
Petit LLM dans le navigateur qui "comprend" chaque endpoint à la volée	non déterministe, lourd, difficile à tester, danger de sémantique inventée
Dupliquer manuellement tous les services backend dans le frontend	coût élevé, dérive inévitable, maintenance double
Rejouer simplement les requêtes HTTP après reconnexion	ne gère pas l'état local, les dépendances, demi-écritures ni conflits
Mettre toute la base en IndexedDB	risque sécurité, quota, coût de sync, données inutiles
CRDT partout	ne convient pas à tous les invariants et effets métier
Background Sync comme mécanisme obligatoire	support navigateur insuffisant; cycle de vie non garanti
Last-write-wins universel	perte silencieuse de données et violation d'invariants

27. Conclusion

La forme industrialisable de l'idée est plus proche d'un compilateur de comportement distribué que d'un modèle IA embarqué. L'IA conserve un rôle stratégique : comprendre le code, générer une représentation sémantique candidate, produire des tests et réduire le travail d'annotation. Mais la sécurité, la cohérence et l'exécution doivent rester déterministes et prouvables.

La proposition la plus forte d'AERIS est donc : “les développeurs conçoivent normalement leur application; la plateforme détermine et compile le sous-ensemble du backend qui peut être reproduit localement, maintient les données minimales nécessaires, exécute ce sous-ensemble dans le navigateur et réconcilie avec le serveur sans compromettre l'autorité serveur.”

Recommandation de départ

Construire d'abord un prototype vertical NestJS + PostgreSQL + React avec 5-10 endpoints, shadow execution et tests différentiels. Ne pas commencer par un LLM navigateur ni par le support multi-framework. La première preuve à obtenir est : “même entrée + même fixture -> même résultat et mêmes effets métier pour le sous-ensemble supporté”.

Annexe A - Schéma conceptuel de l'AERIS IR

```
AerisArtifact := {
  artifactVersion, runtimeMinVersion, sourceCommit, signature,
  endpoints: EndpointPlan[],
  projections: Projection[],
  migrations: LocalMigration[],
  policies: PolicySet
}

EndpointPlan := {
  route: {method, path},
  inputSchema, outputSchema,
  authRequirements,
  dataDependencies,
  program: Instruction[],
  offlineClass, freshnessPolicy,
  syncContract, conflictPolicy,
  evidence: SourceEvidence[],
  testVectorRefs[]
}

Instruction :=
  QUERY | ASSERT | COMPUTE | INSERT | UPDATE | DELETE |
  EMIT_LOCAL_EVENT | QUEUE_INTENT | RETURN
```

A.1 Versioning

Chaque artefact contient une version du format IR, une version métier, le commit source backend et la version minimale du runtime. Une modification de schéma local ou de sémantique sync doit produire une nouvelle version. Le runtime refuse un artefact dont il ne comprend pas la version.

A.2 Evidence

Chaque règle générée doit idéalement pointer vers des preuves : fichier/ligne/symbole, contrainte DB, annotation ou test. Cette traçabilité est essentielle pour revue, audit et débogage du compilateur.

Annexe B - Protocoles serveur proposés

B.1 Snapshot

```
GET /aeris/snapshot?projection=proj-42
Authorization: Bearer ...
If-None-Match: ...
```

```
200 SnapshotEnvelope
```

B.2 Delta

```
GET /aeris/delta?projection=proj-42&since=981244
-> records[], tombstones[], next_cursor
```

B.3 Reconcile

```
POST /aeris/reconcile
{
  client_instance_id,
  artifact_version,
  operations: [
    {operation_id, endpoint_id, base_cursor, payload, dependencies, preconditions}
  ]
}
-> receipts[] | conflicts[]
```

B.4 Policy manifest

```
GET /aeris/policy
-> endpoint overrides, kill switches, min versions, TTL overrides
```

Annexe C - Checklist d'éligibilité d'un endpoint

Question	Oui	Non -> action
Tous les appels sont résolus statiquement ?	continuer	UNSUPPORTED ou annotation
Les effets externes sont absents ou modélisés ?	continuer	ONLINE_REQUIRED
Les données nécessaires peuvent être projetées sans fuite ?	continuer	ONLINE_REQUIRED/refactor
Les validations sont déterministes sur snapshot ?	LOCAL candidate	SPECULATIVE ou ONLINE
L'effet serveur est idempotent ?	continuer	ajouter idempotency contract
Les conflits ont une politique explicite ?	continuer	ne pas activer écriture offline

Question	Oui	Non -> action
Les tests différentiels passent ?	publish candidate	bloquer CI
La taille de projection respecte le budget ?	activer	réduire projection/pagination

Annexe D - Glossaire

Terme	Définition
Artifact	ensemble versionné et signé des plans AERIS publiés pour une application
Behavior Compiler	outil CI qui extrait et compile la sémantique supportée du backend
Behavior IR	représentation intermédiaire exécutable du comportement d'un endpoint
Projection	sous-ensemble local et autorisé des données serveur nécessaire à certains endpoints
Snapshot	état cohérent d'une projection à un curseur donné
Delta	ensemble de changements depuis un curseur
Outbox	journal durable des intentions locales non encore confirmées serveur
Operation ID	identifiant stable d'une intention logique, base de l'idempotence
Commit Receipt	réponse serveur attestant qu'une opération est devenue canonique
Conflict Package	réponse structurée expliquant pourquoi une opération ne peut être committée telle quelle
Speculative	exécution locale utile à l'UX mais pouvant être invalidée au retour réseau
Shadow execution	exécution locale parallèle en online utilisée uniquement pour vérifier la parité

Références techniques sélectives

- R1. MDN - Service Worker API.** https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
- R2. MDN - Background Synchronization API (compatibilité limitée).**
https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API
- R3. MDN - IndexedDB API.** https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
- R4. MDN - StorageManager / persist / estimate.** <https://developer.mozilla.org/en-US/docs/Web/API/StorageManager>
- R5. MDN - Web Locks API.** https://developer.mozilla.org/en-US/docs/Web/API/Web_Locks_API
- R6. SQLite - WebAssembly / OPFS persistence.** <https://sqlite.org/wasm/doc/tip/persistence.md>
- R7. OpenAPI Specification 3.1.1.** <https://spec.openapis.org/oas/v3.1.1.html>
- R8. Tree-sitter - incremental parsing.** <https://tree-sitter.github.io/tree-sitter/>
- R9. CodeQL - data flow analysis for JavaScript/TypeScript.** <https://codeql.github.com/docs/codeql-language-guides/analyzing-data-flow-in-javascript-and-typescript/>
- R10. W3C - Trace Context.** <https://www.w3.org/TR/trace-context/>
- R11. MDN - WebAssembly.** <https://developer.mozilla.org/en-US/docs/WebAssembly>